

php

Développement Web

Introduction à PHP

PHP est un langage de programmation populaire en développement Web : il assure le fonctionnement de presque 50% des sites Internet.

Les scripts PHP sont exécutés par le serveur qui héberge le site. C'est le résultat de l'exécution, souvent du contenu HTML, qui est envoyé au navigateur client.

On parle alors de langage server-side ou de back-end : il est de fait associé aux systèmes de gestion de base de données, MySQL ou MariaDB.

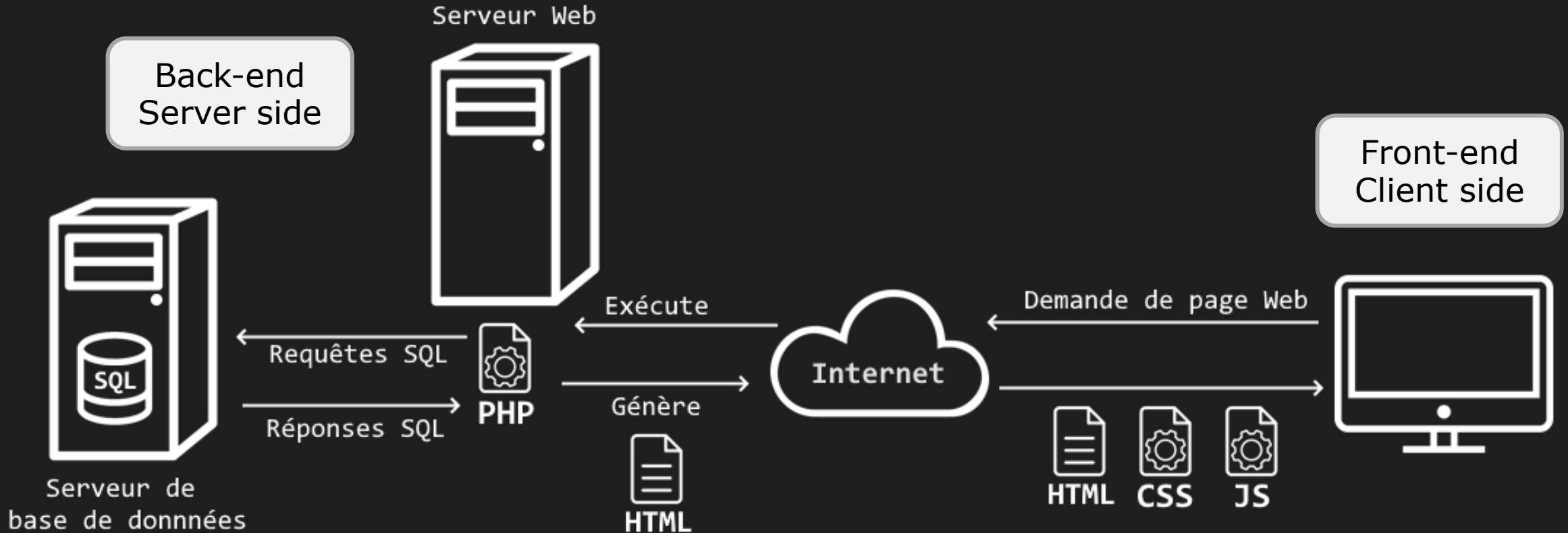
C'est un logiciel open source, gratuit et multi-plateforme :

- OS : Linux, Windows...
- Serveur Web : Apache, IIS, Nginx...

Il est en version 8.4. Fortement concurrencé par d'autres langages, PHP a évolué, il prend en charge par exemple le typage fort.

Introduction à PHP

Lorsqu'un utilisateur demande une page Internet, c'est toujours le même processus.



Le navigateur Web n'a donc jamais accès au fichier PHP.

Installation

Développement Web
php

Installation de PHP

PHP est disponible pour :

- Windows avec le logiciel WAMP, qui regroupe le serveur Apache, MySQL et PHP.

<https://www.wampserver.com>

- MacOS avec le logiciel MAMP, qui regroupe le serveur Apache, MySQL et PHP

<https://www.mamp.info>

- Les distributions Linux :

`apt/dnf install php`

`php -S localhost:7180`

À exécuter dans le répertoire où
se trouve les fichiers PHP

Il est possible d'installer pour les systèmes Linux le serveur Apache (`httpd`) et MySQL (`mysql-server`).

Où doivent se trouver les fichiers PHP ?

Lorsque vous créez un site Web avec un serveur web (Apache, MySQL/MariaDb, PHP), les fichiers doivent être dans un emplacement précis.

Pour Windows avec WAMP, le répertoire racine est : `c:\wamp64\www`

Pour MacOS avec MAMP, le répertoire racine est : `Applications/MAMP/htdocs`

Pour les distributions Linux avec un serveur Web, le répertoire racine est : `/var/www/html`

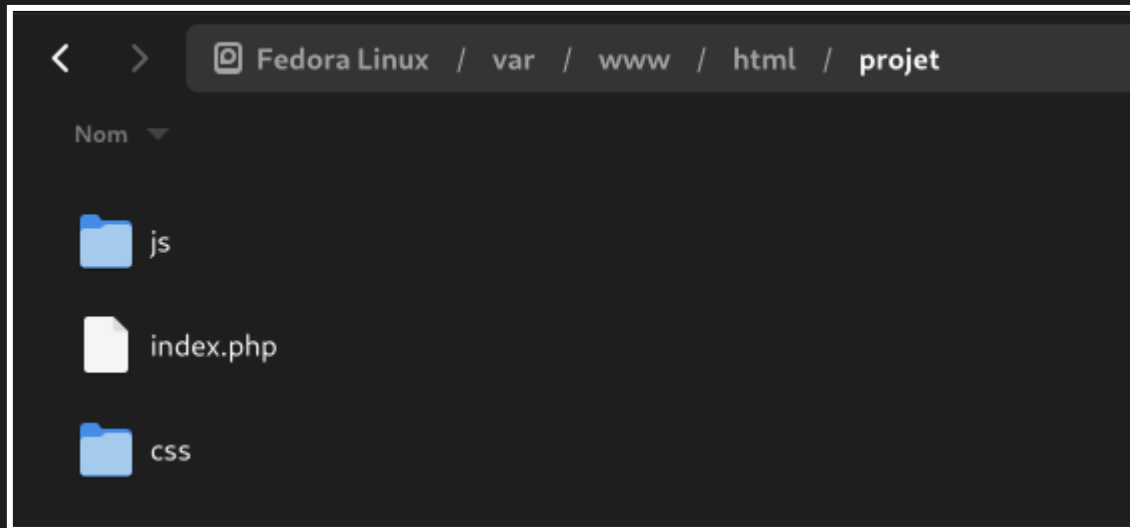
À L'intérieur du répertoire racine vous pouvez créer des dossiers et des sous-dossiers.

Comment accéder à mon site web ?

Ouvrez votre navigateur Internet (Chrome, Firefox, Safari, Edge...) et entrez l'URL selon votre installation : <http://localhost:7180/projet/index.php>

Le numéro de port doit être modifié

Le fichier **index.php** est dans le dossier **projet**, lui-même dans le répertoire racine



Utilisation avec HTML

Développement Web
php

Utiliser PHP dans un document HTML

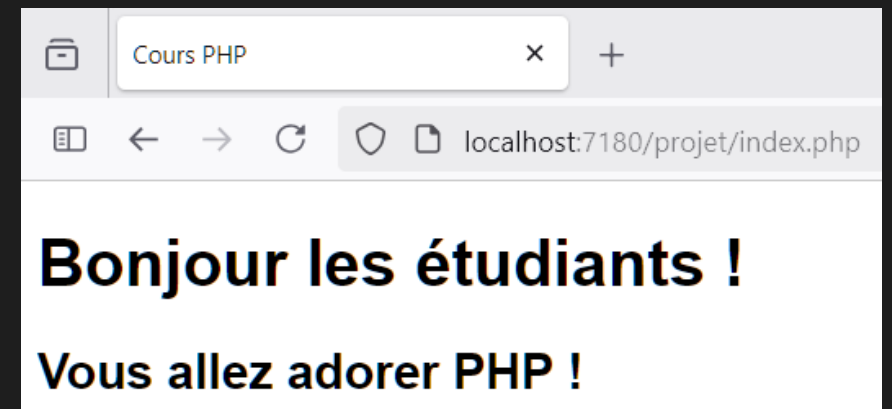
À partir de maintenant vos pages HTML devront avoir l'extension **.php**.

Il y aura dans vos fichiers du code PHP mélangé avec du code HTML.

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Cours PHP</title>
</head>
<body>
<?php echo "<h1>Bonjour les étudiants !</h1>"; ?>
<?= "<h2>Vous allez adorer PHP !</h2>"; ?>
</body>
</html>
```

Une grande section de code PHP sera encadrée par les balises **<?php** et **?>**

La balise **<?=** est un raccourci pour **<?php echo**

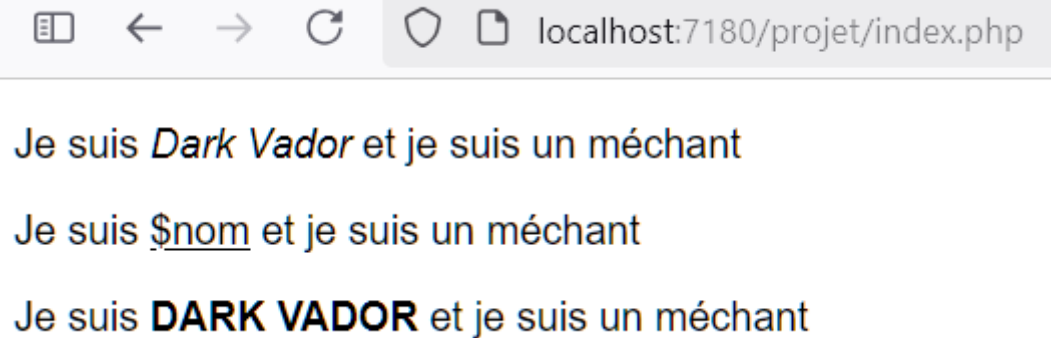


L'instruction **echo**

L'instruction **echo** permet d'ajouter du contenu dans le flux HTML envoyé au navigateur.

Il y a une petite subtilité dans le comportement en fonction du caractère utilisé pour encadrer une chaîne de caractère.

```
<body>
<?php
$nom = "Dark Vador";
echo "<p>Je suis <i>$nom</i> et je suis un méchant<p>";
echo '<p>Je suis <u>$nom</u> et je suis un méchant<p>';
echo "<p>Je suis <b>".strtoupper($nom)."</b> et je suis un méchant</p>";
?>
</body>
```



Je suis *Dark Vador* et je suis un méchant
Je suis \$nom et je suis un méchant
Je suis **DARK VADOR** et je suis un méchant

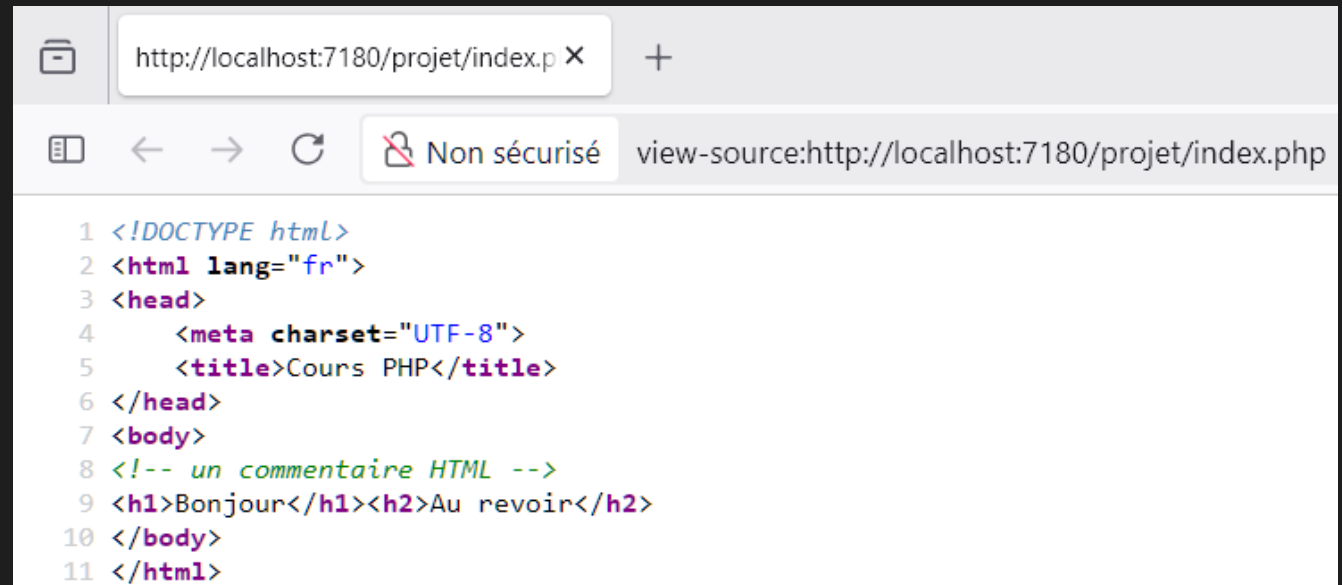


Le **.** est l'opérateur de concaténation pour les chaînes de caractères

Les commentaires

Il est possible de mettre des commentaires, attention au langage utilisé à l'endroit où se trouve le commentaire.

```
<body>
<!-- un commentaire HTML -->
<?php
echo "<h1>Bonjour</h1>";
//un commentaire de ligne
/*
    un autre sur plusieurs lignes
*/
?>
<h2>Au revoir</h2>
</body>
```



```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <title>Cours PHP</title>
6 </head>
7 <body>
8 <!-- un commentaire HTML -->
9 <h1>Bonjour</h1><h2>Au revoir</h2>
10 </body>
11 </html>
```

Les variables

Développement Web
php

L'initialisation et la déclaration des variables

Les variables en PHP :

- Commencent par le symbole **\$**
- Puis une lettre minuscule ou majuscule ou le caractère **_**
- Puis des lettres minuscules ou majuscules, le caractère **_** ou des chiffres.

Il n'est pas nécessaire d'indiquer le type d'une variable, le typage est implicite.

```
<?php
$inconnue;
$texte = "Bonjour";
$entier = 33;
$decimal = 15.24;
$booléen = true;
$tableau = ["Hello", 19, false, [$entier, $decimal]];
?>
```

Exercices

Prérequis : avoir installé PHP.

1. Créez un document PHP :

- Avec la bonne structure HTML
- Un message « *Hello World* », dans une balise `<h2>`, ajouté par l'instruction `echo`.

2. Modifiez le travail précédent, le message « *Hello World* » doit être stocké dans une variable.

Les types primitifs (ou intégrés)

- bool : pour les booléens, **true** ou **false**
- int : pour les nombres entiers
- float : pour les valeurs décimales
- string : pour les chaînes de caractères
- array : pour les tableaux
- object : pour les instances de classe

<https://www.php.net/manual/fr/language.types.php>

Le typage est implicite, c'est PHP qui choisit le type d'une variable au moment de son affectation.

Le typage est aussi dynamique, une variable peut changer de type en cours d'exécution :

```
$variableQuiContientUnNombre = 26;  
$variableQuiContientUnNombre = "vingt-six";
```



Pas d'erreur

La conversion de type dans une opération

PHP peut effectuer des conversions implicites (*transtyper* ou *caster*).

```
$un = "1";  
$deux = 2;  
$trois = $un + $deux;
```

\$trois est égal à l'entier 3

```
$presqueDeux = 1.99;  
$presqueCinq = $presqueDeux + $trois;
```

\$presqueCinq est égal au décimal 4,99

```
$vrai = true;  
$quatre = $trois + $vrai;
```

\$quatre est égal à l'entier 4

```
$faux = false;  
$probablementQuatre = $quatre + $faux;
```

\$probablementQuatre est égal à l'entier 4

Les opérateurs

PHP propose nativement des fonctions liées aux variables :

Opérateurs	Description
<code>+, -, *, /, %</code>	Opérateurs mathématiques
<code> , or, &&, and, !</code>	Opérateurs logiques
<code>==, !=</code>	Comparaisons
<code>>, <, >=, <=</code>	Comparaisons de valeurs mathématiques
<code>===, !==</code>	Comparaisons de valeurs et de types
<code>++, --</code>	Incrémentation et décrémentation
<code>+=, -=, /=, *=, %=</code>	Modification de valeurs mathématiques
<code>., .=</code>	Concaténations
<code>??</code>	Affectation contrôle de valeur

Quelques fonctions liées aux variables

PHP propose nativement des fonctions liées aux variables :

- `isset($var)` : retourne **true** si la variable a été initialisée
- `unset($var)` : détruit la variable
- `is_numeric($var)` : retourne **true** si la variable contient une valeur numérique
- `is_bool($var)` : retourne **true** si la variable contient une valeur booléenne
- `is_array($var)` : retourne **true** si la variable contient un tableau
- `intval("192.33")` : retourne la valeur numérique entière équivalente d'une variable

Il existe d'autres fonctions qui permettent de déterminer le type d'une variable ou de convertir une variable vers un autre type :

<https://www.php.net/manual/fr/ref.var.php>

Un peu de maths

PHP propose nativement des fonctions liées aux variables :

```
$a = 17;  
$c = $a + 4;  
$c = $a / 3;
```

```
$c += 9;  
$c /= 3;  
$c *= 3;
```

Reviens à faire l'affectation de l'opération
sur la variable : `$c = $c / 3;`

```
$c = round(33 / 4, 2);  
$c = floor(33 / 4);  
$c = ceil(33 / 4);
```

Les fonctions d'arrondi (normal,
inférieur et supérieur)

Quelques fonctions mathématiques

PHP propose nativement des fonctions liées aux variables :

- `pow($valeur, $exposant)` : retourne `$valeur` à la puissance `$exposant`
- `max($valeur1, $valeur2, $valeur3...)` : retourne la valeur la plus grande
- `pi()` : retourne la valeur de Pi
- `rand($min, $max)` : retourne une valeur aléatoire entre `$min` et `$max`
- `abs($valeur)` : retourne la valeur absolue de `$valeur`

Les structures de contrôles

Développement Web
php

La structure conditionnelle `if else`

Une structure conditionnelle permet d'effectuer ou de ne pas effectuer une section de code selon une ou plusieurs conditions.

L'évaluation des conditions se fait dans un test, `if`, qui est considéré comme vrai (`true`) ou faux (`false`).

```
if($age < 18) {  
    echo "Tu es mineur";  
}  
else {  
    echo "Tu es majeur";  
}
```

Ce bloc n'est réalisé que si la variable `$age` contient un entier strictement inférieur à 18.

Si la condition n'est pas remplie, alors on rentre dans le bloc `else`

Si l'élément à évaluer n'est pas un booléen, il y aura une conversion implicite.

if, else if, else

Il est possible d'enchaîner plusieurs conditions avec **else if**.

```
if($pays == "usa" and $age >= 21) {  
    echo "Adult";  
}  
else if($pays == "france" and $age >= 18) {  
    echo "Majeur";  
}  
else {  
    echo "Children/Mineur";  
}
```

L'opérateur ternaire ?

Il est possible de remplacer une structure `if else` par l'opérateur ternaire.

```
if($value % 2 == 0) $retour = "Pair";  
else $retour = "Impair";
```

```
$retour = ($value % 2 == 0) ? "Pair" : "Impair";
```



Le résultat est
identique

L'opérateur peut aussi permettre d'appeler une fonction particulière selon un test :

```
(rand(1, 5) > 3) ? doSomething() : dontDoAnything();
```


Évaluation coupe-circuit

Dans le test d'une structure conditionnelle, il peut y avoir plusieurs conditions à évaluer. Ces conditions sont séparées par des opérateurs logiques (voir la page « Les opérateurs »).

Si **\$age** est supérieure ou égale à 21, on exécute le code attaché au **if**, sans évaluer la suite du test (opérateur **or**)

```
if($age >= 21 or ($pays == "france" and $age >= 18)) {  
    echo "Tu es majeur";  
}
```

Si **\$pays** est égale à « france », on doit tout de même vérifier que la condition d'âge est respectée (opérateur **and**)

La conversion de type dans une structure conditionnelle

La conversion de type peut aussi intervenir dans les structures de contrôle comme **if**.

`if(0)`
`if("0")`
`if('0')`

if non déclenché, conversion vers l'entier `0` puis vers le booléen `false`

`if("")`
`if('')`

if non déclenché, conversion d'une `String` vide vers le booléen `false`

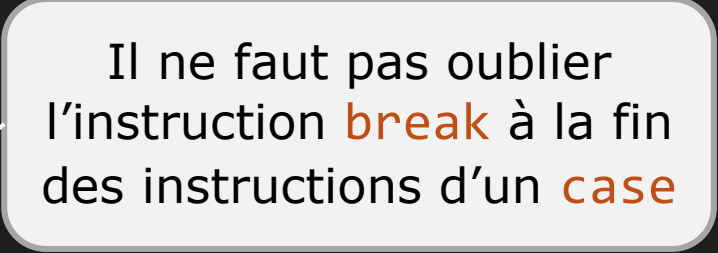
`if("00")`
`if('00')`
`if(" ")`

if déclenché, conversion d'une `String` non vide vers le booléen `true`

Le bloc `switch case`

Dans le cas où une variable déclenche des comportements différents selon sa valeur, on peut utiliser un bloc `switch case`.

```
$aleatoire = rand(1, 10);  
switch($aleatoire) {  
    case 1: echo "un est particulier"; break;  
    case 2: echo "deux est un nombre premier"; break;  
    case 3: echo "trois est un nombre premier"; break;  
    case 5: echo "cinq est un nombre premier"; break;  
    case 7: echo "sept est un nombre premier"; break;  
    default:  
        if($aleatoire % 2 == 0) echo "c'est un nombre pair";  
        else echo "c'est un nombre impair";  
        break;  
}
```



Il ne faut pas oublier
l'instruction `break` à la fin
des instructions d'un `case`

Exercices

En reprenant l'exemple de la page précédente et en générant une valeur aléatoire comprise entre 1 et 48 (inclus), déterminez si ce nombre est premier, impair ou pair.

1. Avec un `switch case`
2. Avec une succession de `if else if`

```
1 est un nombre impair.  
2 est un nombre premier.  
3 est un nombre premier.  
4 est un nombre pair.  
5 est un nombre premier.  
6 est un nombre pair.  
7 est un nombre premier.  
8 est un nombre pair.  
9 est un nombre impair.  
10 est un nombre pair.  
11 est un nombre premier.  
12 est un nombre pair.  
13 est un nombre premier.  
14 est un nombre pair.
```

Les chaînes de caractères

Développement Web
php

Quelques fonctions sur les chaînes de caractères

PHP propose nativement des fonctions liées aux chaînes de caractères :

- `strlen($variable)` : retourne le nombre de caractères d'une chaîne
- `strtolower($variable)` : retourne la chaîne de caractères en minuscules
- `strtoupper($variable)` : retourne la chaîne de caractères en majuscules
- `trim($variable)` : retourne la chaîne de caractères sans espace avant et après
- `substr($variable, $starting, $length)` : retourne la sous-chaîne de `$variable` à partir de l'index `$starting` et d'une longueur `$length`
- `str_replace($search, $replace, $variable)` : retourne la chaîne où `$search` est remplacé par `$replace` dans `$variable`
- `explode($delimiter, $string)` : retourne un tableau avec les éléments d'une chaîne, séparés par `$delimiter`

<https://www.php.net/manual/fr/ref.strings.php>

Les boucles

Développement Web
php

Effectuer un traitement en boucle

Il arrive très fréquemment que l'on soit obligé de faire un traitement plusieurs fois en programmation. Il existe 3 structures qui réalisent des boucles :

La boucle **for** pour un traitement où l'on sait en avance le nombre maximum de cycle.

La boucle **while** lorsque le nombre de cycle à effectuer est inconnu, parfois il est égal à 0.

La boucle **do while**, identique à **while** mais au moins un cycle est effectué.

Chaque tour de boucle est une itération. Avant une itération (**for** et **while**) ou après (**do while**), le système vérifie s'il doit continuer à itérer.

La boucle **for**

La boucle **for** est utilisée lorsque le nombre maximum d'itérations est connu avant la boucle.

L'instruction
d'initialisation

La condition préalable
à chaque tour

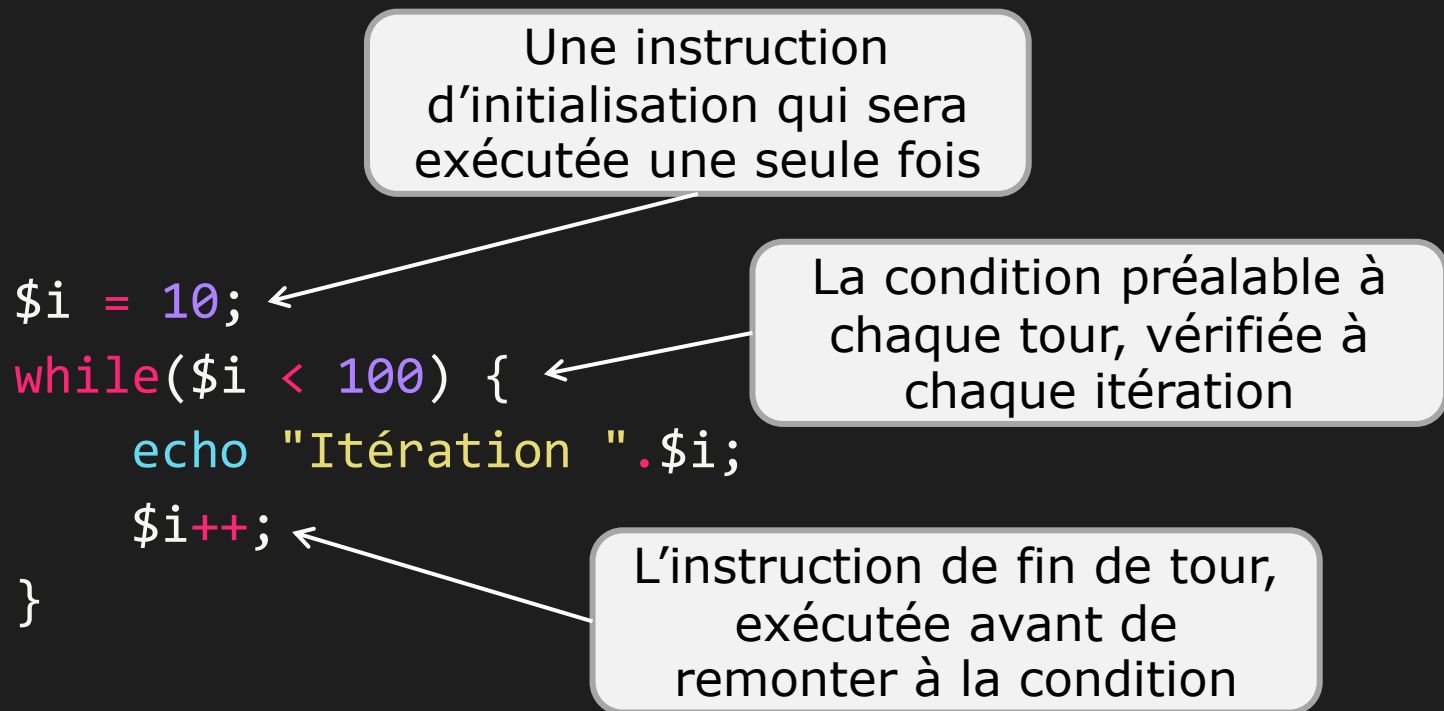
L'instruction de
fin de tour

```
for($i = 1; $i < 10; $i++) {  
    echo "Itération ".$i;  
}
```

La boucle **while**

La boucle **while** est utilisée lorsque le nombre maximum d'itérations n'est pas connu à l'avance.

Une boucle **while** peut remplacer une boucle **for**, mais pas l'inverse.



La boucle `while`

Une boucle `while` peut ne faire aucune itération.

```
$variable = false;  
while($variable) {  
    echo "Une itération";  
}
```

Le message ne
s'affichera jamais

On utilise les boucles `while` surtout pour des traitements où le nombre d'itérations est inconnu : la lecture des résultats d'une requête SQL ou la lecture d'un fichier.

```
$fichier = fopen('texte.txt', "r");  
while(!feof($fichier)) {  
    $ligne = fgets($fichier);  
    echo $ligne;  
}  
fclose($fichier);
```

La condition vérifie que l'on
n'a pas atteint la fin du fichier,
end-of-file

La boucle **do while**

La boucle **do while** est utilisée lorsque l'on ne connaît pas d'avance le nombre de cycles à effectuer, mais on veut faire au moins un tour de boucle.

```
$variable = false;  
$i = 0;  
do {  
    echo "Une itération";  
    $i++;  
} while($variable and $i < 100);
```

← La condition est évaluée
à la fin du tour

Le contrôle des boucles : **break**

L'instruction `break` permet de mettre fin immédiatement à une boucle : on sort du bloc et on continue les instructions qui suivent.

```
$aleatoire = rand(1, 100);
```

`$aleatoire` est une valeur aléatoire choisie entre 1 et 100.

```
for($i = 1; $i <= 100; $i++) {
```

```
    if($i == $aleatoire) {
```

```
        echo "Le nombre mystère est ".$i;
```

```
        break;
```

L'instruction `break` arrête immédiatement la boucle `for`

```
    }
```

```
    echo "Le nombre mystère n'est pas ".$i;
```

```
}
```

```
echo "La boucle est terminée";
```

Le contrôle des boucles : **continue**

L'instruction **continue** arrête l'itération en cours et retourne à la déclaration de la boucle : soit à l'instruction de fin de tour soit à la condition.

```
$aleatoire = rand(1, 50) * 2;  
for($i = 1; $i <= 100; $i++) {
```

\$aleatoire est une valeur aléatoire
paire choisie entre 2 et 100

```
    if($i % 2 == 1) continue;
```

Si **\$i** est impair, on arrête l'itération
actuelle avec **continue** comme si
l'itération s'était terminée normalement.

```
    if($i == $aleatoire) {  
        echo "Le nombre mystère est ".$aleatoire;  
        break;  
    }  
    echo "Le nombre mystère n'est pas ".$aleatoire;  
}
```

Les tableaux

Développement Web
php

Généralités concernant les tableaux

L'utilisation des tableaux avec PHP est simple.

- Lors de la déclaration, on ne doit pas indiquer la taille du tableau
- Le tableau n'a alors pas besoin d'être redimensionné
- L'index peut être numérique (tableau indexé), alphanumérique (tableau associatif) ou mixte
- La gestion d'un index numérique peut être pris en charge par PHP, même avec un index mixte
- Les éléments du tableau peuvent être de tout type, même des tableaux

Un tableau est de type **array**.

Gestion automatique de l'index

Si l'utilisateur ne définit pas de valeur d'index, PHP va gérer un index numérique pour chaque tableau :

- Le premier élément est à l'index 0
- Les éléments suivants seront positionnés à la plus grande valeur numérique +1

Si un élément est ajouté avec un index alphanumérique, il n'y a pas de perturbations de la gestion de l'index.

La valeur de l'index numérique ne définit pas la position de l'élément dans le tableau : si aucun tri n'est effectué, les éléments se suivent dans l'ordre chronologique de l'ajout d'un élément.

Création et ajout d'éléments dans un tableau

```
$tableau = array('Citron', 6 => 'Orange', 'Pomelos');  
$tableau[] = 'Mandarine';  
$tableau[5] = 'Clémentine';  
$tableau[] = 'Bergamote';  
$tableau['agrume1'] = 'Cédrat';  
$tableau[] = 'Lime';  
  
foreach($tableau as $index => $valeur) {  
    echo $index." -> ".$valeur;  
}
```

```
0 -> Citron  
6 -> Orange  
7 -> Pomelos  
8 -> Mandarine  
5 -> Clémentine  
9 -> Bergamote  
agrume1 -> Cédrat  
10 -> Lime
```

Parcourir un tableau

Les boucles **for** et **while** peuvent être utilisées pour parcourir un tableau. Cependant à la différence d'autres langages, l'index peut être mixte et aussi discontinu.

Il est préférable d'utiliser une boucle **foreach**.

```
$tableau[5] = 'Clémentine';
```



```
foreach($tableau as $index => $valeur) {  
    echo $index." -> ".$valeur;  
}
```

La boucle **foreach** parcourt tous les éléments du tableau, les instructions **break** et **continue** peuvent être utilisées.

Quelques fonctions liées aux tableaux

PHP propose nativement des fonctions liées aux tableaux :

- `count($tableau)` : retourne le nombre d'éléments dans le tableau
- `in_array($search, $tableau)` : retourne `true` si `$search` est dans `$tableau`
- `is_array($variable)` : retourne `true` si `$variable` est un tableau
- `implode($delimiter, $tableau)` : retourne une chaîne de caractères avec les éléments du tableau, séparés par `$delimiter`
- `explode($delimiter, $string)` : retourne un tableau avec les éléments d'une chaîne, séparés par `$delimiter`
- `array_merge($tableau1, $tableau2, ...)` : retourne un tableau qui fusionne les tableaux passés en paramètres
- `shuffle($tableau)` : mélange le tableau
- `sort($tableau)` : tri ascendant d'un tableau sur les valeurs
- `ksort($tableau)` : tri ascendant d'un tableau sur l'index

<https://www.php.net/manual/fr/array.sorting.php>

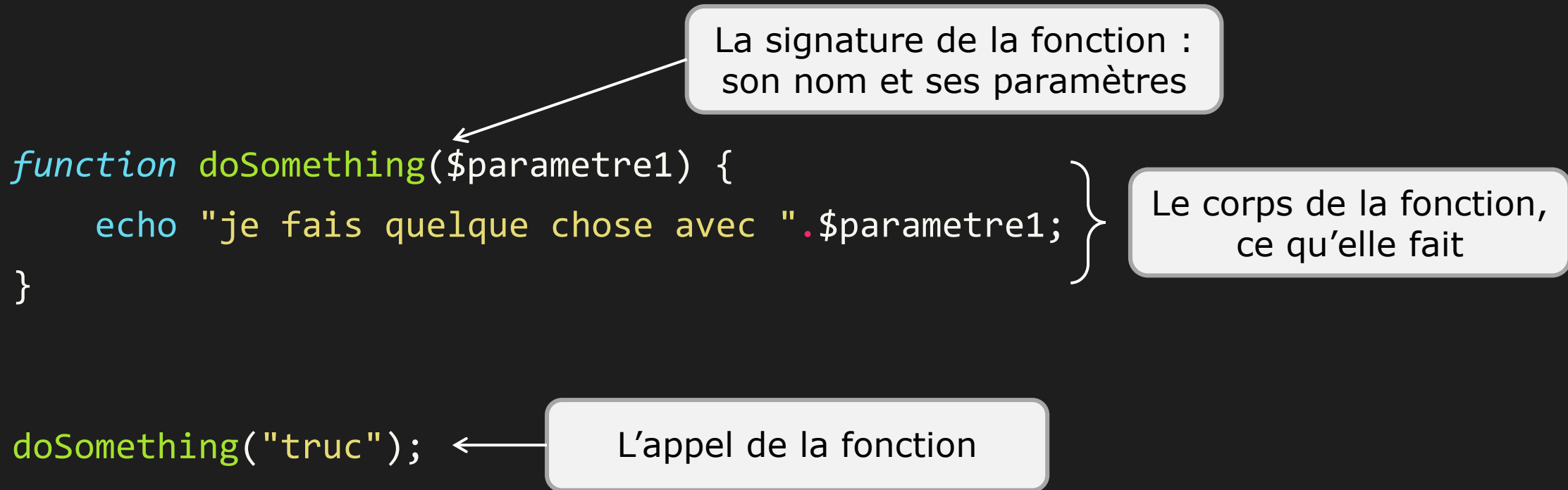
Plusieurs fonctionnements différents

Les fonctions

Développement Web
php

Créer une fonction

La création de fonctions avec PHP repose sur le même processus que dans les autres langages : la déclaration et les appels



Retourner une valeur ou interrompre la fonction

Si nécessaire une fonction peut retourner une information avec le mot-clé `return`. Il peut être utilisé sans valeur, pour interrompre le fonctionnement de la fonction.

```
function returnSomething($parametre1, $parametre2) {  
    if($parametre1 % 2 == 0) return $parametre1 + $parametre2;  
    else {  
        switch($parametre2) {  
            case 0: return;  
            case 1, 3: return $parametre2; break;  
            case 6, 9, 12: return $parametre2 - $parametre1;  
        }  
    }  
    return $parametre2;  
}
```



Les paramètres des fonctions

Une fonction peut avoir plusieurs paramètres, ils sont séparés par des virgules.

Certains paramètres peuvent être optionnels, ils sont placés à la fin et sont suivis de leur valeur par défaut.

Paramètres
obligatoires



```
function doSomething($param1, $param2, $param3 = false, $param4 = 8) { }
```

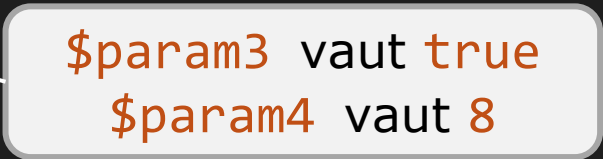
```
doSomething("Bonjour", "Au revoir");
```

\$param3 vaut false
\$param4 vaut 8



```
doSomething("Hello", "Good bye", true);
```

\$param3 vaut true
\$param4 vaut 8



```
doSomething("Yo", "Ciao", false, 9);
```


Le typage fort des fonctions PHP

A partir de PHP 7, il est possible de typer les fonctions : pour les paramètres et pour le type de retour (ou **void** si elle ne retourne rien).

\$param1 doit être
du type **String**

\$param3 doit être du type
bool, elle prend par
défaut la valeur **false**

```
function doSomething(String $param1, int $param2 = 0, bool $param3 = false) : int {  
    return 0;  
}
```

La fonction doit retourner
une valeur typée **int**

Les fichiers

Développement Web
php

Inclure du code dans un fichier PHP

On peut inclure le contenu d'un fichier dans un fichier PHP pour :

- Organiser/séparer du code dans des fichiers distincts
- Réutiliser du code commun dans plusieurs fichiers
- Faciliter le travail en groupe

Le code inclut est identique à du code qui aurait été déjà présent dans le fichier.

4 directives existent, elles ont le même rôle, mais des comportements différents :

Directive	Si le fichier est manquant	Si le fichier a déjà été inclut
<code>include</code>	Warning	Nouvel import
<code>require</code>	Erreur	Nouvel import
<code>include_once</code>	Warning	Directive ignorée
<code>require_once</code>	Erreur	Directive ignorée

Exemple d'inclusion

```
<?php
```

```
require('session.php');
```

Pourrait contenir les fonctions qui chargent la session et les informations de l'utilisateur

```
include('html/head.php');
```

Inclut les instructions HTML de la balise `<head>`, dont les styles CSS et les fichiers JS

```
?>
```

```
<body>
```

```
<?php
```

Cette fonction serait probablement déclarée dans le fichier `session.php`

```
if(isAdmin()) include('menu_admin.php');  
else include('menu_standard.php');
```

```
include 'footer.php';
```

```
?>
```

```
</body>
```

```
</html>
```

Les parenthèses sont optionnelles

Sauvegarder des données

Il existe un grand nombre de formats de fichiers :

- Les formats « propriétaire » où les données stockées ne peuvent être lues que par un programme particulier.
- Les formats « ouverts » où les données sont stockées selon des règles que tout le monde peut utiliser. Les données restent souvent lisibles.

JSON est un format standard ouvert pour transmettre ou sauvegarder des données :

- Plus léger et plus simple que XML
- Plus facile à lire que certains fichiers comme CSV ou TSV
- Très largement supporté par les langages de programmation.

Écrire des données dans un fichier JSON

PHP propose deux fonctions pour écrire des données dans un fichier JSON :

- **file_put_contents** : créé ou écrase un fichier, ajoute des données et ferme le fichier

<https://www.php.net/manual/fr/function.file-put-contents.php>

- **json_encode** : retourne une chaîne qui représente des données JSON

<https://www.php.net/manual/fr/function.json-encode.php>

```
$data = array('nom' => "Citron vert", 3.14159, 42, array("Cédrat", -1), true);
```

```
file_put_contents("data.json", json_encode($data, JSON_PRETTY_PRINT));
```



Le nom du
fichier

Les données à
sauvegarder

Lecture des données d'un fichier JSON

PHP propose deux fonctions pour lire des données d'un fichier JSON :

- **file_get_contents** : retourne toutes les données du fichier

<https://www.php.net/manual/fr/function.file-get-contents.php>

- **json_decode** : retourne les données incluses dans une structure JSON

<https://www.php.net/manual/fr/function.json-decode.php>

```
$backdata = json_decode(file_get_contents("data.json"), true);
```



Une variable qui contiendra
les données récupérées



Le nom du
fichier

Lire et écrire un fichier

L'accès aux fichiers locaux est simple.

```
if(file_exists("data.txt")) {  
    $filestream = fopen("data.txt", 'r');  
    while(!feof($filestream)) {  
        $line = fgets($filestream, 10000);  
    }  
    fclose($filestream);  
}  
else {  
    $filestream = fopen("data.txt", 'w');  
    fwrite($filestream, "Coucou");  
    fclose($filestream);  
}
```

Les valeurs de `fopen()` : <https://www.php.net/manual/fr/function.fopen.php>

Quelques fonctions liées aux fichiers

PHP propose nativement des fonctions liées aux fichiers :

- `file_exists($fichier)` : return `true` si le fichier existe
- `unlink($fichier)` : supprime le fichier
- `copy($original, $destination)` : copie un fichier à un nouvel emplacement
- `rename($fichier, $nouveau)` : renomme un fichier
- `file_get_contents($fichier)` : retourne le contenu entier d'un fichier ou `false`

<https://www.php.net/manual/fr/ref.filesystem.php>

Les spécificités de PHP pour le web

Développement Web
php

PHP en développement Web

PHP doit sa réputation à son rôle dans le développement Web.

Le code PHP est exécuté par le serveur Web et dispose de fonctionnalités qui sont liées au monde du Web :

- Lecture des paramètres de l'URL
- Lecture des informations transmises par les formulaires
- Création des sessions et des cookies
- Redirection des pages Web

Un projet Web étant principalement mis à disposition des utilisateurs sur Internet, il y a une conséquence sur l'aspect sécurité.

Modifier une page Web avec PHP

Un bloc de code PHP peut se trouver dans une section HTML avec les balises `<?php` et `?>`. C'est le résultat de l'exécution qui sera ajouté au flux HTML.

```
<html>
<body>
  <h2>
    <?php
      if(date("G") > 17) echo "Bonne soirée";
      else if(date("G") > 12) echo "Bon après-midi";
      else if(date("G") == 12) echo "Bon appétit";
      else echo "Bonjour";
    ?>
  </h2>
</body>
</html>
```

Les particularités du fonctionnement Web

Parce que le code PHP est exécuté sur le serveur et qu'il génère un flux HTML/CSS, à la fin de l'exécution toutes les variables sont supprimées, comme lorsqu'un programme, en C ou en Java, s'arrête.

D'une page à l'autre, par un lien hypertexte ou par un formulaire, l'état du système (les variables) doit être recréé à partir de 0.

Il existe des outils qui permettent de :

- Passer des informations
- Conserver des variables

Le tableau associatif `$_GET`

Le tableau associatif `$_GET` est créé automatiquement PHP. Il est composé de couples « Clé-valeur » qui sont déclarés dans l'URL.

<http://localhost/get.php?firstname=Marie&lastname=Curie&birthdate=1867-11-07>

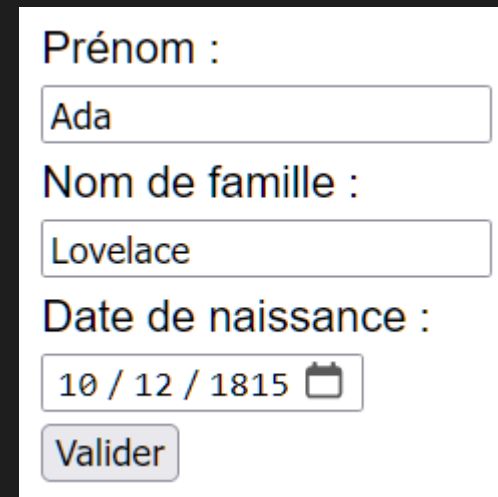
```
<?php
foreach($_GET as $key => $value) {
    echo '<br>$_GET'."['$key'] == ".$value;
}
if(isset($_GET['firstname'])) {
    echo "<br>Bonjour ".$_GET['firstname'];
}
?>
```

```
$_GET['firstname'] == Marie
$_GET['lastname'] == Curie
$_GET['birthdate'] == 1867-11-07
Bonjour Marie
```

Le tableau associatif `$_POST`

Le tableau associatif `$_POST` est créé automatiquement PHP. Il est composé de couples « Clé-valeur » qui sont déclarés dans un formulaire HTML.

```
<form action="http://localhost/post.php" method='POST'>
    <label for="prenom">Prénom : </label>
    <input type="text" id="prenom" name="firstname">
    <label for="nom">Nom de famille : </label>
    <input type="text" id="nom" name="lastname">
    <label for="naissance">Date de naissance : </label>
    <input type="date" id="naissance" name="birthdate">
    <input type="submit" value="Valider">
</form>
```



Prénom :
Ada

Nom de famille :
Lovelace

Date de naissance :
10 / 12 / 1815 

Valider

```
<?php
foreach($_POST as $key => $value) {
    echo '<br>$_POST'."<br>['$key'] == ".$value;
}
?>
```

```
$_POST['firstname'] == Ada
$_POST['lastname'] == Lovelace
$_POST['birthdate'] == 1815-12-10
```

Le tableau associatif `$_REQUEST`

Il arrive qu'une information soit passée par l'URL ou par un formulaire. Elle peut donc se trouver dans `$_GET` ou dans `$_POST`.

PHP propose un autre tableau associatif, `$_REQUEST`, qui regroupe les données de `$_GET` et `$_POST`.

```
<?php
foreach($_REQUEST as $key => $value) {
    echo '<br>$_REQUEST' . "['$key'] == ".$value;
}
?>
```



```
$_REQUEST['firstname'] == Katherine
$_REQUEST['lastname'] == Johnson
$_REQUEST['birthdate'] == 1918-08-26
```

Si une clé existe dans l'URL et dans le formulaire avec des valeurs différentes, c'est la valeur dans `$_POST` qui sera maintenue dans `$_REQUEST`.

Les sessions et le tableau `$_SESSION`

Les sessions permettent de sauvegarder des données d'un utilisateur sur le serveur.

Ce n'est pas une sauvegarde à long terme :

- Un cookie est automatiquement stocké sur le poste client pour mémoriser l'identifiant de la session
- La session a une durée d'expiration après la dernière interaction
- La session n'est pas transférable d'un navigateur à un autre

On ne stockera donc dans `$_SESSION` que des données temporaires que le système pourra régénérer en cas d'expiration de la session :

- Panier,
- Historique de visite,
- Réglages...

Démarrer une session

L'identifiant de la session, qui permet au serveur de retrouver les données, est stocké dans un cookie.

L'utilisation du cookie impose que la session soit démarrée avant l'envoi du flux HTML.

```
<?php session_start(); ?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Document</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Bonjour <?= $_SESSION['username'] ?? "anonyme"; ?>!</h2>
```

```
</body>
```

```
</html>
```

PHP va créer ou récupérer les données de la session

On peut alors utiliser des données du tableau associatif `$_SESSION`

Ajouter des données dans une session

L'ajout des données dans une session est très simple.

```
<?php session_start(); ?>
<!DOCTYPE html>
<html>
<head><!-- --></head>
<body>
    <?php
        $_SESSION['timestamp'][] = date('U');
        foreach($_SESSION['timestamp'] as $key => $value) {
            echo "<br>$value";
        }
    ?>
</body>
</html>
```

Le tableau `$_SESSION` est associatif
et peut avoir plusieurs dimensions
(des sous-tableaux)

Dans cet exemple, à chaque fois que la
page est visualisée, le système ajoute la
valeur de l'horodatage (*timestamp*)
dans la session

Quelques fonctions liées aux sessions

PHP propose nativement des fonctions liées aux sessions :

- `session_start()` : crée ou récupère la session
- `session_id()` : définit ou retourne l'identifiant de la session
- `session_status()` : retourne la valeur de l'énumération (voir documentation PHP)
- `session_destroy()` : détruit toutes les données associées à la session, sans pour autant supprimer le cookie.
- `session_unset()` : détruit toutes les variables de la session.

<https://www.php.net/manual/fr/ref.session.php>

Les cookies et le tableau `$_COOKIE`

Les cookies permettent de sauvegarder des données d'un utilisateur dans les fichiers du navigateur.

- Un cookie est automatiquement stocké sur le poste client, il contient des données
- Les informations d'un cookie peuvent être lu par le code Javascript
- Le site web choisira la durée de vie du cookie, cependant le navigateur peut détruire les cookies à la fermeture de la page.

On ne stockera donc dans `$_COOKIE` que des données temporaires que le système pourra régénérer en cas d'expiration de la session et dont la confidentialité n'est pas nécessaire.

Les données dans `$_COOKIE` sont créées à la réception des informations par le serveur, puis le code PHP est exécuté. La fonction `setcookie()` n'ajoute pas les données à `$_COOKIE`, ces données ne seront disponibles qu'à la prochaine page.

Envoyer et lire un cookie

L'utilisation des cookies impose qu'ils soient transmis avant l'envoi du flux HTML.

```
<?php
setcookie("username", "Anne O'Nyme", time() + 60);
?>
<!DOCTYPE html>
<html>
<head><!-- --></head>
<body>
    <h2>Bonjour <?= $_COOKIE['username'] ?? "anonyme"; ?>!</h2>
</body>
</html>
```

Création d'un cookie avec une durée de vie de 60 secondes

Lecture d'un cookie

Erreurs communes avec les cookies

- Les cookies ne seront accessibles qu'au chargement de la prochaine page, ou au rechargement de la page courante.
- Il faut éviter de définir la valeur d'un cookie à **false**.
- La date d'expiration d'un cookie doit être définie par rapport au nombre de secondes depuis le 1^{er} janvier 1970, que l'on peut obtenir avec **time()**.

`setcookie("username", "Pouet", time() - 60);` ← Suppression du cookie

`setcookie("username", "Truc", 0);` ← Le cookie sera supprimé à la fermeture de la page

`setcookie("username", "Machin", 86400);` ← Le cookie expire dans un jour

<https://www.php.net/manual/fr/function.setcookie.php>

La fonction `header()`

Cette fonction permet d'envoyer des informations en entête de requête, comme pour les cookies et les sessions, elle doit être utilisée avant le flux HTML.

Elle très utile pour rediriger sur une autre page :

```
<?php header("Location: https://www.php.net/"); ?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Document</title>
```

```
</head>
```

```
<body>
```

```
    <p>Cette page ne s'affichera jamais !</p>
```

```
</body>
```

```
</html>
```


La gestion des erreurs

Les exceptions

Développement Web
php

try {} catch {}

Certaines opérations peuvent générer des erreurs qui ne peuvent pas être « anticipées » par le programme, ces erreurs sont nommées des exceptions.

- Une connexion a une base de données a échoué,
- Un fichier n'a pas pu être ouvert,
- Une division par zero,
- Une variable qui contient `null`...

```
try {  
    fonctionCapricieuse();  
}  
catch(Exception $e) {  
    echo "fonctionCapricieuse() a levé une exception : ".$e->getMessage();  
}
```

Cette fonction peut générer
une exception

Pour obtenir le message
d'erreur de l'exception

try {} catch {} finally {}

Le mot-clé `finally` permet de créer un bloc de code qui s'exécute que l'on soit passé dans un `catch` ou pas.

Il est particulièrement utile si plusieurs `catch` interceptent des exceptions différentes.

```
try {  
    fonctionCapricieuse();  
    echo "fonctionCapricieuse n'a pas levé une exception";  
}  
catch(OutOfRangeException $e) { }  
catch(DivisionByZeroError $e) { }  
catch(Exception $e) { }  
finally {  
    echo "quoiqu'il arrive on passe ici";  
}
```

On peut gérer des exceptions
de types différents

throw

Parfois on peut vouloir jeter une erreur pour des cas précis.

```
function getAge($anneeNaissance) {  
    if($anneeNaissance < 1900) throw new Exception("Trop vieux");  
    return date('Y') - $anneeNaissance;  
}
```